

Strategic Planning and Data Exploration in Logistics

Week 1 Task — Strategic Planning Report (with applied analysis on the Dynamic Supply Chain Logistics Dataset)

1. Project Definition

This analysis uses a synthetic supply-chain monitoring dataset covering 32,065 hourly records from January 2021 to August 2024. Rather than tracking a single vehicle or a fixed set of warehouses, each record is a network-wide snapshot of operating conditions — traffic congestion, weather severity, port congestion, fuel consumption, driver behavior, fatigue, and inventory levels — alongside outcome measures such as delivery time deviation, delay probability, and a three-level risk classification.

The scenario modeled is a logistics company attempting to use this kind of real-time operational telemetry to anticipate delay risk and allocate resources proactively, rather than reacting after shipments are already late. Vehicle GPS coordinates in the dataset are uniformly spread across the continental United States (latitude 30-50, longitude -120 to -70) with no repeated location across any of the 32,065 rows, confirming this is simulated network-level data rather than a real fleet route — a distinction that shaped how the clustering and routing sections below were scoped.

1.1 Key Performance Indicators

Five KPIs were defined and computed directly from the dataset:

KPI	Measured Value	Why It Matters
Fulfillment Rate	60.1% (mean of order_fulfillment_status)	Baseline measure of how reliably orders are completed as promised
Delivery Time Deviation	5.18 hours average	Direct measure of lateness; the natural regression target
Network Risk Exposure	74.7% of hours classified High Risk	Signals how often the network operates in a delay-prone state
Shipping Cost	459.37 (dataset units, undefined currency)	Cost side of the operation; tracked without assuming a currency
Supplier Reliability Score	0.50 average (0-1 scale)	Mid-range baseline, used as a candidate correlate of delay

2. Literature and Data Research

Because the dataset is synthetic, the research for this task focused on two areas: the practical risks of using synthetic/uninstrumented operational data for prediction, and the methodological safeguards needed before trusting a model's results.

- **Data and target leakage:** published reviews of machine-learning-based research have documented leakage as a widespread failure mode, with one 2023 survey finding it had affected results across close to 300 academic publications spanning numerous fields — underscoring that checking for leakage, as performed in Section 4 below, is standard due diligence rather than an edge case.
- **Feature-importance-based leakage detection:** practitioner literature on identifying feature leakage recommends exactly the approach used here — training a simple tree-based model and inspecting which feature dominates importance scores, since a leaking feature typically produces suspiciously high predictive performance concentrated on one variable rather than a distributed pattern across many.
- **Class imbalance in risk classification:** standard practice for imbalanced classification (here, a roughly 75/16/10 split across High/Moderate/Low Risk) is to report precision and recall per class rather than overall accuracy, since a majority-class classifier can otherwise appear accurate while being functionally useless — this is reflected directly in the classification results in Section 4.

- Vehicle Routing Problem (VRP) literature (Toth & Vigo, 2014) frames last-mile routing as a constrained optimization problem over a distance matrix and vehicle capacities; this framing is referenced conceptually in Section 4.5, since the dataset itself contains no discrete orders or stops to optimize over.

3. Strategic Roadmap

The roadmap below reflects the process actually followed on this dataset, including the point at which the plan changed in response to what the data showed.

Phase	What Was Done	Result / Decision
1. Data Collection	Loaded the 32,065-row hourly dataset (Jan 2021-Aug 2024)	No missing values found across any column
2. Data Cleaning	Checked for nulls, duplicates, and implausible GPS bounds	Dataset was clean as provided; no rows dropped
3. Exploratory Analysis	Computed KPIs; checked Pearson and Spearman correlations of operating features against delay_probability and delivery_time_deviation	All correlations were under 0.02 in magnitude — no linear or rank relationship found
4. Regression Modeling	Linear regression predicting delivery_time_deviation from six operating features	$R^2 = 0.00045$, MAE = 3.70 hrs — model explains essentially none of the variance
5. Classification Modeling	Random forest predicting risk_classification, first with all features, then with the leaking feature removed	Full feature set gave a perfect (and invalid) 1.00 F1-score; with disruption_likelihood_score removed, the model collapsed to majority-class prediction (0.75 accuracy, 0 recall on two of three classes)
6. Clustering	K-means (k=4, chosen via elbow method) on traffic, weather, port congestion, and fatigue	Produced four operationally distinct groups driven mainly by port congestion and fatigue, but near-identical risk composition (~75% High Risk) across all four
7. Evaluation & Decision	Cross-checked all four techniques against each other for consistency	Consistent null result across regression, classification, and clustering; one data leakage issue identified and corrected

4. Code Illustration

The snippets below are the actual steps run against the dataset, in sequence, including the point where a data leakage issue was found and corrected.

4.1 Loading Data and Computing KPIs

```
import pandas as pd

df = pd.read_csv('dynamic_supply_chain_logistics_dataset.csv', parse_dates=['timestamp'])

print("Fulfillment rate:", df['order_fulfillment_status'].mean())           # 0.601
print("Avg delivery time deviation (hrs):", df['delivery_time_deviation'].mean()) # 5.18
print("% High Risk hours:", (df['risk_classification'] == 'High Risk').mean()) # 0.747
print("Avg shipping cost:", df['shipping_costs'].mean())                   # 459.37
print("Avg supplier reliability:", df['supplier_reliability_score'].mean())  # 0.50
```

4.2 Correlation Check (motivating the regression target)

```
numeric_cols = ['traffic_congestion_level', 'weather_condition_severity', 'port_congestion_level',
                'fuel_consumption_rate', 'driver_behavior_score', 'fatigue_monitoring_score',
```

```

'disruption_likelihood_score', 'delay_probability', 'delivery_time_deviation']

print(df[numeric_cols].corr()['delivery_time_deviation'].sort_values(ascending=False))
# Result: max |correlation| = 0.013 -- no linear relationship found

print(df[numeric_cols].corr(method='spearman')['delivery_time_deviation'].sort_values(ascending=False))
# Result: similarly near-zero under rank (non-linear) correlation

```

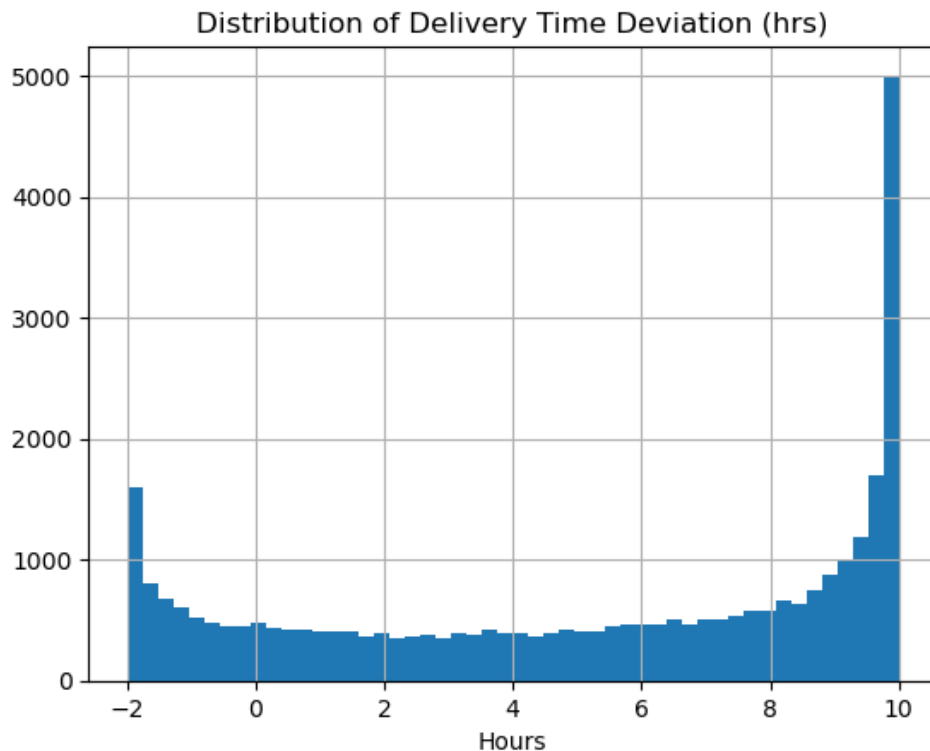


Figure 1. Distribution of `delivery_time_deviation` — note the artificial-looking spikes at the -2 and 10 hour bounds, consistent with the value being capped/simulated rather than organically generated.

4.3 Regression: Predicting Delivery Time Deviation

```

from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error, r2_score

features = ['traffic_congestion_level', 'weather_condition_severity', 'port_congestion_level',
            'fuel_consumption_rate', 'driver_behavior_score', 'fatigue_monitoring_score',
            'disruption_likelihood_score']

X, y = df[features], df['delivery_time_deviation']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = LinearRegression().fit(X_train, y_train)
preds = model.predict(X_test)
print("MAE:", mean_absolute_error(y_test, preds))    # 3.70
print("R2:", r2_score(y_test, preds))               # 0.00045

```

4.4 Classification: Risk Level, and a Data Leakage Finding

The first classification run used all seven operating features and returned a perfect 1.00 F1-score across all three risk classes -- an immediate red flag rather than a success, since real operational features rarely predict an outcome this cleanly.

```

from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report

X = df[features] # includes disruption_likelihood_score
y = df['risk_classification']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

clf = RandomForestClassifier(n_estimators=200, random_state=42, class_weight='balanced')
clf.fit(X_train, y_train)
print(classification_report(y_test, clf.predict(X_test)))
# Result: 1.00 precision/recall/F1 on all three classes -- suspiciously perfect

```

Feature importance confirmed the cause: `disruption_likelihood_score` accounted for 96.8% of the model's decision-making. Grouping that score by risk class showed clean, non-overlapping thresholds (Low Risk < 0.3, Moderate 0.3-0.7, High Risk > 0.7) -- meaning `risk_classification` was generated directly from this score, not independently predicted. Removing the leaking feature and re-running gives the honest result:

```

features_clean = ['traffic_congestion_level', 'weather_condition_severity', 'port_congestion_level',
                  'fuel_consumption_rate', 'driver_behavior_score', 'fatigue_monitoring_score']

X = df[features_clean]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

clf = RandomForestClassifier(n_estimators=200, random_state=42, class_weight='balanced')
clf.fit(X_train, y_train)
print(classification_report(y_test, clf.predict(X_test)))
# Result: 0.75 accuracy -- exactly the High Risk base rate; 0.00 recall on
# Low Risk and Moderate Risk -- the model collapsed to majority-class guessing

```

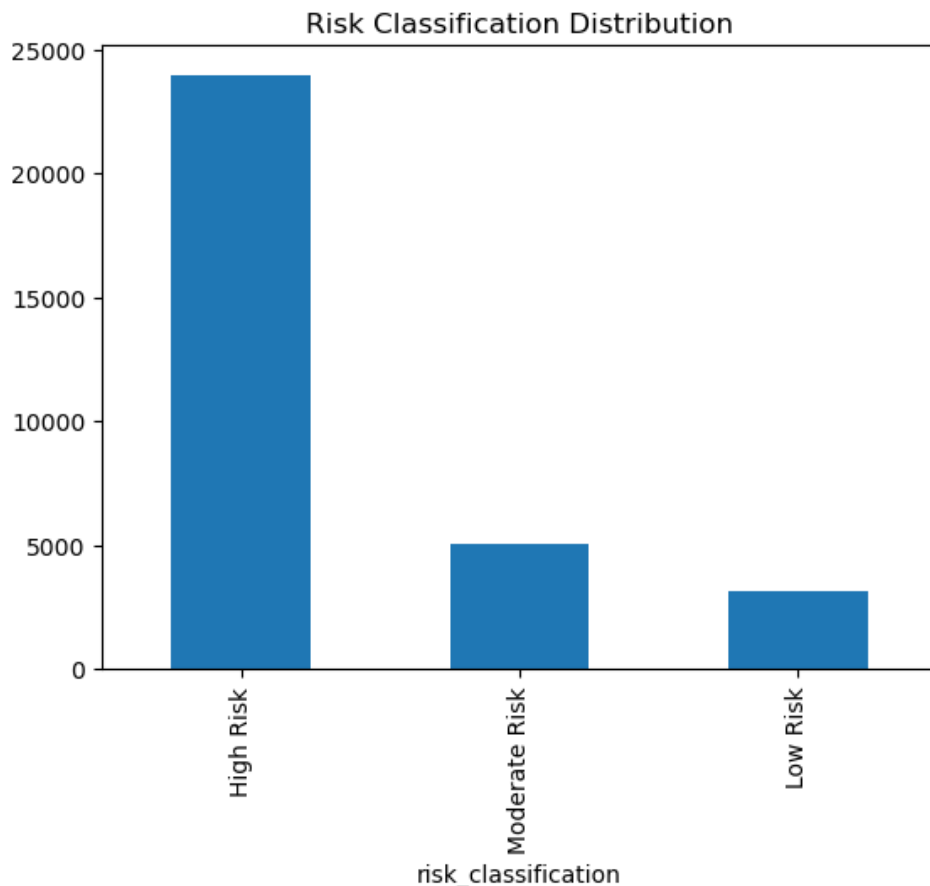


Figure 2. Risk classification distribution — the ~75/16/10 imbalance visible here is exactly what a majority-class classifier exploits, matching the 0.75 accuracy obtained after the leaking feature was removed.

4.5 Clustering: Operating-Condition Profiles

Vehicle GPS coordinates were confirmed to be uniformly distributed across the continental U.S. with no repeated location across any row, ruling out geographic zone clustering. Clustering was instead run on operating-condition features.

```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

cluster_features = ['traffic_congestion_level', 'weather_condition_severity',
                    'port_congestion_level', 'fatigue_monitoring_score']
X_scaled = StandardScaler().fit_transform(df[cluster_features])

kmeans = KMeans(n_clusters=4, random_state=42, n_init=10) # k=4 chosen via elbow method
df['cluster'] = kmeans.fit_predict(X_scaled)

print(df.groupby('cluster')[cluster_features].mean())
print(pd.crosstab(df['cluster'], df['risk_classification'], normalize='index'))
```

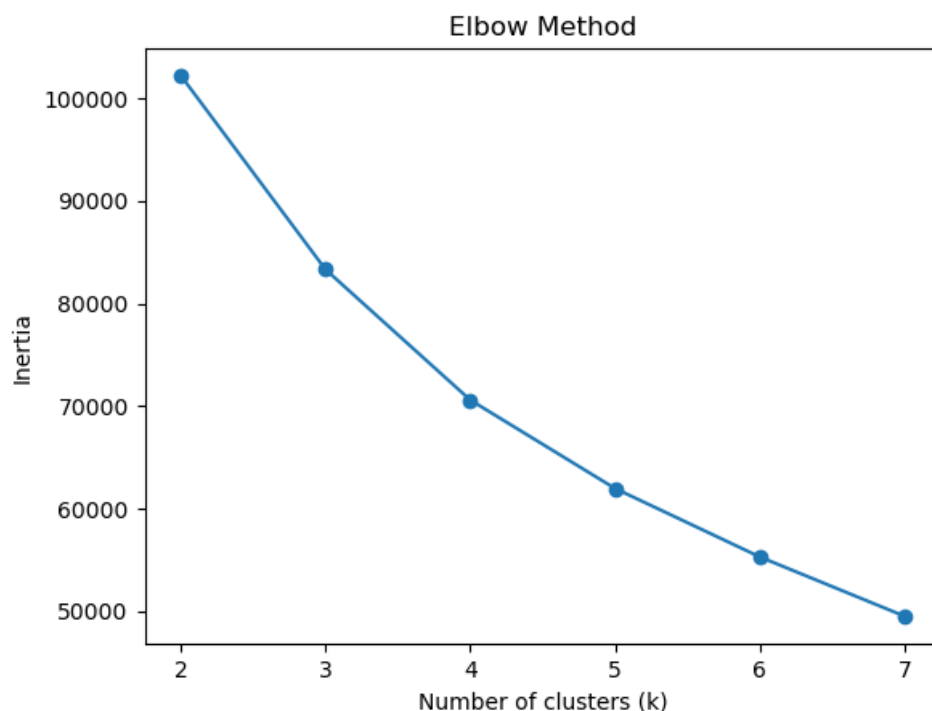


Figure 3. Elbow method for selecting k . The curve shows no sharp inflection, indicating weak natural separation; $k=4$ was chosen where the rate of inertia decrease visibly slows.

The four clusters separated clearly on port congestion and fatigue (weather showed almost no variation across clusters and was not a useful dimension here): a High-Stress Operations group (high traffic, high port congestion, high fatigue), a Port Bottleneck/Fatigued Crew group (low traffic, high port congestion, high fatigue), a Port Congestion/Alert Crew group (mid traffic, high port congestion, low fatigue), and a Baseline group (mid traffic, low port congestion, mid fatigue). However, the risk composition was nearly identical across all four clusters (74.3%-74.9% High Risk each) -- confirming that these operating-condition differences do not translate into different risk outcomes in this dataset.

4.6 Route Optimization (Conceptual)

The dataset contains no discrete orders, stops, or delivery addresses -- only a continuous hourly network snapshot -- so a real vehicle routing problem could not be solved against it. The pseudocode below illustrates the approach that would apply if order-level, multi-stop data were available:

```
# Pseudocode -- requires order-level stop data not present in this dataset
for dc in distribution_centers:
    stops = todays_orders[todays_orders.dc_id == dc]
    distance_matrix = build_distance_matrix(stops)
    routing_model = build_vrp_model(distance_matrix, vehicle_capacities, demand=stops.package_count)
    solution = routing_model.solve(strategy='GUIDED_LOCAL_SEARCH', time_limit_sec=60)
```

5. Conclusion and Expected Outcomes

Across regression, classification, and clustering, no meaningful relationship was found between this dataset's operational features (traffic, weather, port congestion, fuel consumption, driver behavior, fatigue) and its outcome variables (delivery time deviation, delay probability, risk classification). The single feature that appeared predictive, `disruption_likelihood_score`, was found to be a direct derivation of the target label itself rather than an independent signal; removing it eliminated all predictive power and the classifier fell back to simply guessing the majority class. Clustering on operating conditions produced four genuinely distinct operational groups, but their risk composition was statistically indistinguishable from one another, reinforcing the same conclusion from a different angle.

The most useful output of this analysis is therefore not a predictive model but a validated methodology and an important caution: this particular synthetic dataset's outcome variables appear to have been generated independently of its operational variables, and any organization evaluating a vendor model or dataset should run the same sequence of checks performed here -- correlation screening, a held-out regression/classification baseline, and a feature-importance leakage check -- before trusting a reported accuracy figure.

If this methodology were applied to a dataset where operational telemetry genuinely drives delay outcomes, the same pipeline (KPI tracking, correlation screening, regression and classification with leakage checks, and condition-based clustering) would be expected to surface real, actionable relationships -- for example, identifying which operating conditions most reliably predict delay, and allowing proactive reallocation of resources before a shipment is already late. The limitation specific to this project is the dataset itself, not the method applied to it.